

DBD281

Project

Group C

Aidan Lakmeeharan

Abongile Tyekela

Enrique De Sousa

Thierry Suleman

Contents

Title Page.....	1
Table of Contents.....	2
Introduction.....	3
Physical Layout.....	3
ERD Conceptual Model.....	4
Normalization.....	5
Logical Design.....	6
Queries.....	6

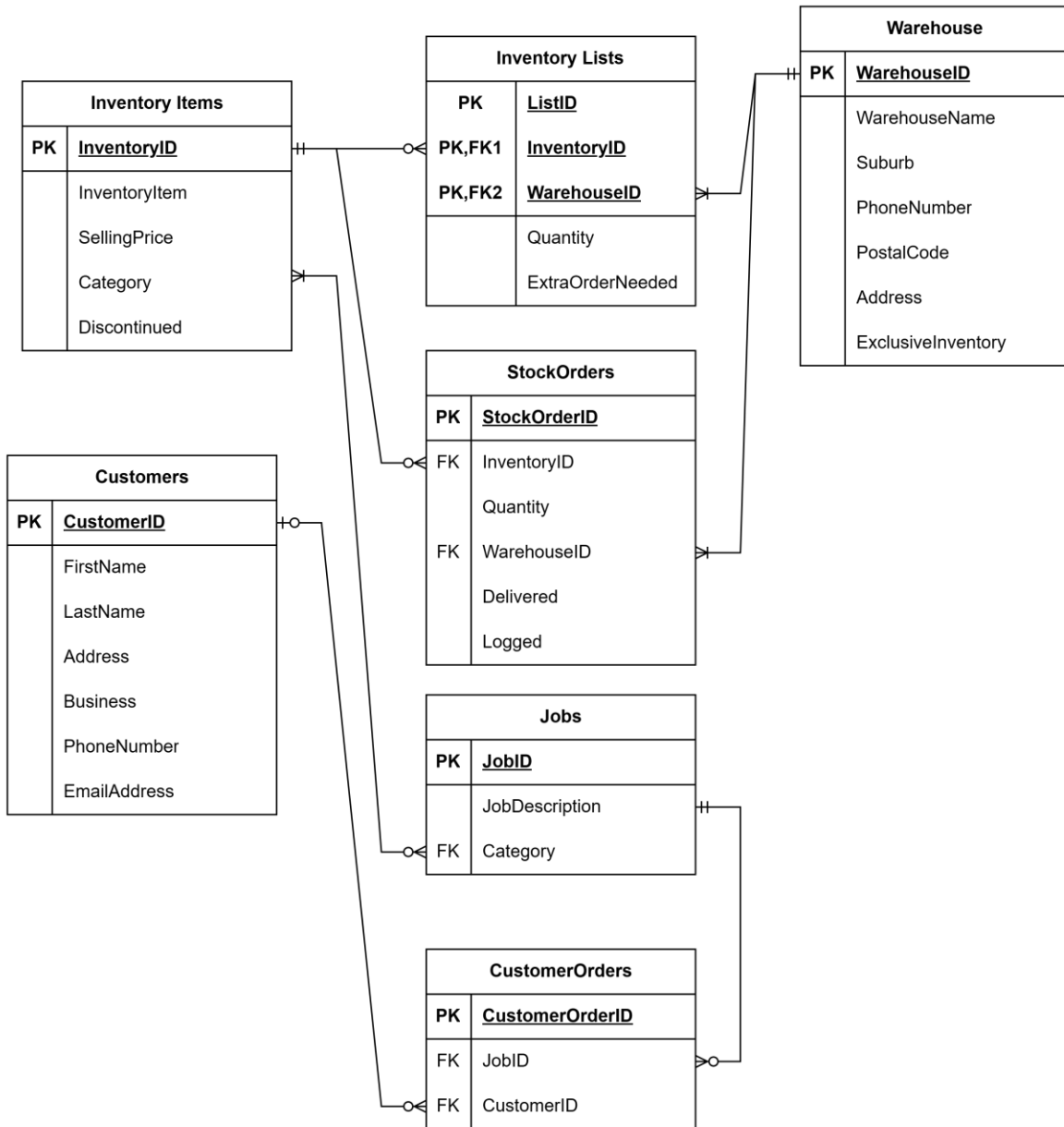
Introduction

Brainy Holdings is an IT, Security and Solar company specializing in Networking, Camera security and Solar Power for both household and business use. After consulting with their CEO and founder, it has been discovered that there is a substantial issue with their inventory management. The company is not efficient nor effective at tracking inventory across their warehouses and walk-in store for customers. They are not able to keep track of stock in the warehouses, nor do they know where the stock is even if they know they have stock on hand. This also leads to rising travel costs and time wasting as they need to travel to each warehouse to confirm where or even if they have inventory for certain jobs. We believe a database solution can assist with these issues, as a centralized database can keep track of inventory in each warehouse, its quantity, as well as maximize efficiency when jobs require inventory from specific warehouses or inventory must be purchased.

Physical Design

The database proposed for Brainy Holdings will be hosted on an on-Site Server at the Brainy Office. There will be a backup of the Database stored on separate devices in each warehouse as well as the Brainy Office. Connecting to these servers will not be an issue for any database administrators as the Brainy Office and warehouses have reliable, high-speed Fiber internet connectivity. To access the database, the Software Microsoft SQL Server will need to be installed on the user's devices. User's devices must be powerful enough to run the database; however, these devices are already in use for Brainy Holdings, specifically devices with 16-32GB of RAM and at most a 3-generation old Intel i5 or AMD Ryzen 5 CPU. Server access and database access will be password protected and encrypted.

ERD Conceptual Model



Normalization

Understanding the data

The data we need to store is

- CustomerID, CustomerOrderID, InventoryID, ListID, WarehouseID, CategoryID, StockOrderID
- Inventory item, Price, Category, Discontinued
- Warehouse Name, Warehouse Suburb, Warehouse Phone Number, Warehouse Postal Code, Warehouse Address, Exclusive Inventory
- Quantity of the items, a tick box if extra orders is needed
- FirstName, LastName, Address, Business, PhoneNumber, EmailAddress
- JobDescription

We need to establish our entities in our database

- Customers
- CustomerOrders
- StockOrders
- Jobs
- InventoryItems
- InventoryLists
- Warehouse

Justifications on why the database meets the requirements for third Normal Form (3NF)

In the current database we have created eliminates all the repeating group and ensures that each column contains only one value. Establishing First Normal Form (1NF). In the Second Normal Form (2NF) the database must have no partial dependencies which was ensured by 1NF and all the non – key attributes that is in the table is dependent on the primary key. In the database we have removed all the transitive dependencies which was already completed in the 2NF which meets the requirements for Third Normal Form (3NF)

Logical Design

Customers table

- CustomerID – Primary Key, Number
- FirstName – Varchar, this is the first name of the customer
- LastName – Varchar, this is the last name of the customer
- Address – Varchar, this is the address of the customer
- Business – Yes/No, this is to check if the customer is involved with a business
- PhoneNumber – Varchar, this is the contact of the customer
- Email Address – Varchar, this is to contact the customer via email

Inventory Items

- InventoryID – Primary Key, Number
- InventoryItem – Varchar, this is the name of the item in the inventory
- SellingPrice – Money, this is the selling price of the item
- Category – Varchar, this is for the category of the item
- Discontinued – Yes/No, checks if the item is discontinued or not

Inventory Lists

- ListID – Primary Key, Number
- InventoryID – Foreign Key, Number
- WarehouseID - Foreign Key, Number
- Quantity – Number, this is for the number of items in the inventory
- ExtraOrderNeeded – Yes/No, this is to check if there is more orders needed for the item

Warehouse

- WarehouseID – Primary Key, Number
- WarehouseName – Varchar, this is the name of the warehouse that holds the items
- Suburb – Varchar, this is the suburb where the warehouse is located
- Phonenumner – Varchar, this is the phone number of the warehouse
- PostalCode – Number, postal code of the location the warehouse
- Address – Varchar, this is the address of the warehouse
- ExclusiveInventory – Yes/No, this is to check if the warehouse has specific items

StockOrders

- StockOrderID – Primary Key, Number
- InventoryID – foreign Key, number
- Quantity – Number, this is the number of items that needs to be bought
- WarehouseID – foreign Key, Number
- Delivered – Yes/No, this is to check if the stock is delivered or not
- Logged – Yes/No, this is to check if the stock was logged

Jobs

- JobID – Primary Key, Number
- JobDescription – Varchar, this is the description of the job that is identified
- Category – Foreign Key, Varchar, this is for the category of the item

CustomerOrders

- CustomerOrderID – Primary Key, Number
- JobID – Foreign Key, Number
- CustomerID - Foreign Key, Number

Queries used in this database

This query calculates the average selling price of the inventory items where the category is technology:

```
SELECT AVG(SellingPrice) AS [Average Price]
FROM InventoryItems -- Table name
WHERE Category = 'Technology'
```

This query displays the suburbs that has more than 1 warehouse:

```
SELECT *
FROM Warehouse
WHERE Suburb IN (
    SELECT Suburb
    FROM Warehouse
    GROUP BY Suburb
    HAVING COUNT(*) > 1
);
```

This query joins inventory items, jobs and the customer tables together:

```
SELECT c.FirstName, c.LastName, j.JobDescription, i.InventoryItem, i.Category
FROM Customers c
```

INNER JOIN CustomerOrders co ON c.CustomerID = co.CustomerID

INNER JOIN Jobs j ON co.JobID = j.JobID

INNER JOIN InventoryItems i ON j.Category = i.Category

This query selects all the customers where they are a part of a business:

SELECT FirstName, Address, PhoneNumber, EmailAddress

FROM Customers

WHERE Business = 'TRUE'

Selects all the inventory items if they are discontinued:

SELECT InventoryID, InventoryItem, SellingPrice, Category

FROM InventoryItems

WHERE Discontinued = 'TRUE'